

Why do Record/Replay Tests of Web Applications Break?

Mouna Hammoudi, Gregg Rothermel
University of Nebraska - Lincoln, USA
{mouna,grother}@cse.unl.edu

Paolo Tonella
Fondazione Bruno Kessler—IRST, Italy
tonella@fbk.eu

Abstract—Software engineers often use record/replay tools to enable the automated testing of web applications. Tests created in this manner can then be used to regression test new versions of the web applications as they evolve. Web application tests recorded by record/replay tools, however, can be quite brittle; they can easily break as applications change. For this reason, researchers have begun to seek approaches for automatically repairing record/replay tests. To date, however, there have been no comprehensive attempts to characterize the causes of breakages in record/replay tests for web applications. In this work, we present a taxonomy classifying the ways in which record/replay tests for web applications break, based on an analysis of 453 versions of popular web applications for which 1065 individual test breakages were recognized. The resulting taxonomy can help direct researchers in their attempts to repair such tests. It can also help practitioners by suggesting best practices when creating tests or modifying programs, and can help researchers with other tasks such as test robustness analysis and IDE design.

I. INTRODUCTION

Web application developers frequently employ record/replay tools to enable the automated testing of their applications. A record/replay tool for web applications captures inputs and actions (mouse clicks, keyboard entries, navigation commands, etc.) that occur as a web application is utilized. During playback, these inputs and actions are re-delivered to the browser engine. The importance of such tools is underscored by the number that exist in the research and commercial realms, including CasperJS [6], CoScripter [23], Dolos [5], Jalangi [39], Mugshot [29], Sahi [36], Selenium [37], Sikuli [46], Test-Maker Object Designer [34], UFT (formerly Quick Test Professional) [41], WaRR [2], and Watir [38].

An advantage of creating web application tests via record/replay tools involves the ability to reuse them to regression test the web applications as they evolve. However, web application tests created by record/replay tools can be quite brittle in the face of system evolution. Changes as simple as renaming a page element or altering the choices in a drop-down list can cause such tests to break and require repair.

For this reason, researchers [9], [13] have recently begun to create techniques for automatically repairing the input and action sequences (hereafter referred to simply as “tests”) created for web applications by record/replay tools. Other research [21], [22], [45] has focused on improving the robustness of such tests. The authors of these papers have acknowledged the brittleness of tests and have singled out particular test constructs that are particularly problematic. There have been

no comprehensive attempts, however, to characterize—via observations of actual evolving web applications—the causes of *test breakages*,¹ much less to measure the frequencies at which those different causes occur. Understanding these issues is arguably a prerequisite for test repair, as well as for other important tasks such as creating maintainable tests and designing record/replay tools.

In this work, we take steps to provide this understanding. We began by collecting 300 versions of five open-source web applications and constructed test suites for the earliest versions of these applications using a record/replay tool. We then iteratively applied the tests to subsequent versions of each application, collecting data on the causes of the failures observed. Given test breakages on a given version V , we then repaired broken tests and augmented the test suite so that repaired tests and tests of new functionality could be carried forward to the version following V . This process yielded data on 722 individual test breakages.

Using the data we gathered, we developed a taxonomy of the causes of test breakages that categorizes all of the breakages observed on the applications we studied. We then gathered 153 versions of three additional web applications and applied the foregoing process to them as well; this yielded data on 343 additional test breakages. We analyzed these in light of our taxonomy and were able to accommodate all of them without further changing the taxonomy; this provides evidence that our taxonomy may be more generally applicable.

Our taxonomy (which to our knowledge is the first taxonomy of web test breakages) is useful in several ways. First, it can help direct the efforts of researchers interested in test repair and help them prioritize their efforts in favor of more common breakage types. Second, it can help engineers avoid the need for test repairs by alerting them to changes that may break them. Third, it can inform the design of automated techniques for detecting problematic constructs in tests and replacing these with less brittle constructs. Fourth, it can inform the design of better IDEs for creating web applications, such as IDEs that help engineers avoid or prevent

¹We define a *test breakage* as the event that occurs when a test that used to function on a web application ceases to be applicable to a new version of that application, due not to a change in an application’s specification (which might render a test “obsolete”), but rather, due to a change that causes the test to no longer function correctly. While such a test can be said to “fail”, the term “test failure” could too easily conflate test breakages with cases in which tests operate correctly and reveal faults in the system under test.

Full Name

Housing ☒ I live on campus

Gender ☐ Male ☐ Female

Car Brand

Developed by [M.Hammoudi](#)

Fig. 1. Student car registration form

test breakages, or automatically repair tests as application modifications are made. Finally, our taxonomy can inform the design of test record/replay tools themselves.

II. BACKGROUND

A. Record/Replay Tools

Record/replay tools allow test engineers to capture sequences of inputs and actions applied to a web application's GUI. The recording process creates a test script that can then be replayed on the application.

As an example, see Fig. 1. The figure depicts the user interface for one form in a web application used to register a car on a college campus, and includes (from top to bottom) a text field, a checkbox, a pair of radio buttons, a dropdown list, a submit button and a link. The code for this web page consists of HTML and Javascript and is shown in Fig. 2. Not visible in the GUI but present in the code is a Javascript alert that causes a popup box to appear if the user attempts to submit an incomplete form.

To further illustrate the record/replay process, Table I shows a test created by Selenium IDE via the application of a sequence of inputs and actions to the registration web page. Each input or action causes a *Selenese command* to be inserted into the test script. Each Selenese command is denoted by a tuple: $\langle \text{action}, \text{locator}, \text{value} \rangle$.² The action component of a Selenese command indicates either an event that is performed on the user interface (e.g. click, select) during the recording process, or an action specific to Selenium's control of the replay process. The locator component specifies the interface element (input field, dropdown list, etc.) that the user is interacting with during that step of the recording process. The value component refers to any input entered by the user within the specified locator, such as a value selected by the user within a dropdown list or a value typed in a text field.

B. HTML Elements, Attributes, and Text

HTML pages are composed of *HTML elements* such as text fields, radio buttons, checkboxes, dropdown lists, and links. HTML elements can have attributes that are known as *name/value pairs* (e.g., `id=FullName`), that provide data on the elements and allow developers to customize them.

²Selenium itself uses the terms "command", "target" and "value"; we choose to use other terminology for clarity and generality.

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title> Student Information Form</title>
5 <script type="text/ javascript">
6   function validateForm() {
7     var FullName = document.forms["sForm"]["FullName"].value;
8     if (FullName == null || FullName == "") {
9       alert ("Please Fill All Required Fields");
10      return false;
11    } else {
12      document.getElementById("myForm").submit();
13    }
14  }
15 </script>
16 </head>
17 <body>
18 <form id="sForm" name="sForm" action="Form2.html" onsubmit="return_validateForm()" method="post">
19   <div>
20     <b>Full Name</b>
21     <input type="text" name="FullName" id="FullName" class="student">
22     <br>
23     <br>
24     <b>Housing</b>
25     <input type="checkbox" name="Living" value="Living" checked="checked"> I live on campus
26     <br>
27     <b>Gender</b>
28     <input type="radio" name="sex" value="male"> Male
29     <input type="radio" name="sex" value="female"> Female
30     <br>
31     <br>
32     <b>Car Brand</b>
33     <select name="cars">
34       <option value="Mercedes">Select Option</option>
35       <option value="Volvo">Volvo</option>
36       <option value="Saab">Saab</option>
37       <option value="Fiat">Fiat</option>
38       <option value="Audi">Audi</option>
39     </select>
40     <br>
41     <br>
42     <input type="submit" value="Submit">
43   </div>
44 </form>
45 <div>
46 <p><a href="http://cse.unl.edu/~mouna"> Developed by M.Hammoudi </a>
47 </p>
48 </div>
49 </body>
50 </html>

```

Fig. 2. HTML for the student car registration form in Fig. 1

TABLE I
A SELENIUM TEST

#	Action	Locator	Value
1	open	~/mouna/example.html	
2	click	css=*:checked	
3	click	document.sForm.elements[2]	
4	select	document.sForm.cars	label=Volvo
5	click	css=form#sForm>div> input[type="submit"]	
6	assertAlert	Please Fill All Required Fields	
7	type	id=FullName	John Smith
8	click	//html/body/form/div/input[5]	
9	clickAndWait	link=Developed by M.Hammoudi	

Attributes can do many things: among these, they can provide an identifier or name for an element, or specify visual characteristics of an element such as its width, height, or color. Attributes can also specify the "dynamic status" of an object; for instance, whether it should be enabled or disabled, or whether a checkbox should be checked by default.

We use the term "element text" to refer to the HTML textual content corresponding to an element that is visible in the UI of a web application. For instance, in Fig. 1, an example of element text is the link text visible at the bottom of the page, "Developed by M.Hammoudi".

C. Locators

Locators are used by JavaScript and other languages, and by record/replay tools, to identify and manipulate elements. We identify two classes of locators, the second of which is composed of two sub-classes.

Attribute-based locators make use of element attributes such as element ids and names to identify elements, or reference element text in order to do so. For instance, in Table I, Line 7 uses an element id and Line 9 uses element text.

Structure-based locators rely on the structure of a web page to identify elements. Among these, *hierarchy-based locators* specify elements in terms of their position in the DOM tree, using strategies involving xpaths, relative xpaths, or CSS selectors. For instance, in Table I, Lines 2, 3, 4, 5, and 8 use structure-based locators. *Index-based locators* operate in situations where multiple elements in a web page have locators that are identical under other strategies; the indices differentiate them. For instance, in Table I, Lines 3 and 8 use index-based locators. As Lines 3 and 8 show, the two structural locator types can appear together.

III. EMPIRICAL PROCESS

Our overall strategy for constructing a taxonomy was data-driven and bottom up. We gathered versions of five web applications, used a record/replay tool to create tests for these, and applied these tests, recording all breakages. We used these results to create our taxonomy, and then we repeated the process on three additional web applications to provide an initial assessment of the taxonomy's validity and generality. We now describe the processes we followed to do this.

A. Objects of Study

To create and assess our taxonomy we searched for complex modern web applications being used by numerous people and under active development. Our criteria for selecting applications required that they: (1) have at least 20 versions, (2) involve at least 30,000 lines of code, (3) be installable and executable, (4) have been downloaded at least 5,000 times, and (5) have experienced at least 300 commits.

Table II provides data on the web applications that we selected, including their names, the number of versions (Rel.) we used (discussed further below), the number of lines of code (counted using `cloc`³ and averaged across the versions), the number of downloads and commits listed for them as of the day we selected them, and the number of tests used for them (also discussed later in this section). The first five applications listed in Table II are the ones we used initially to develop the taxonomy (*training set*), while the last three are the ones we utilized after the initial creation of the taxonomy to validate the applicability of the taxonomy on a different set of web applications (*test set*).

`AddressBook` allows users to manage contacts, and organize and download them. `PHPFusion` is a content management system that allows users to create, manage and administer a website without knowledge about web programming. `MyCollaboration` is a collaboration platform that helps users manage customer information and projects. `PHPAgenda` lets users manage calendars, schedule appointments, holidays, and todo lists, and share content with other users. `Joomla` is

TABLE II
OBJECTS OF STUDY

App Name	Rel.	LOC	Downloads	Commits	Tests
PHPFusion	49	256,899	1,605,195	17,987	47
Address Book	79	35,675	126,146	531	44
PHPAgenda	32	43,831	64,605	357	42
MyCollaboration	29	116,345	7,638	15,876	39
Joomla	111	312,978	>50,000,000	628	56
MyMovieLibrary	23	31,324	6,201	315	36
Dolibarr	30	42,010	864,698	487	38
YourContacts	100	64,765	676,543	573	57

a content management system that allows users with little experience in web programming to publish content. `Dolibarr` is used by companies for enterprise resource planning and customer relationship management. `YourContacts` is used by small and large companies to manage their contacts. `MyMovieLibrary` is used by movie rental stores to organize and track available movies.

`MyCollaboration` is written in Java, `Dolibarr` in Python, and the other applications are written in PHP. All applications use JavaScript, HTML, MySQL, and CSS.

B. Choice of Record/Replay Tool

To conduct our study we needed to choose a representative record/replay tool. As noted in Section I there are many such tools available. Ultimately we chose Selenium IDE, given its popularity, its open-source nature (which will support future work), and the features it supports.

Selenium IDE is in fact similar to many other record/replay tools in terms of the types of information and processes it uses to support replay. Other record/replay tools such as Watir [38], CasperJS [6], and Sahi [36] have test structures that utilize `<action, locator, value>` tuples and utilize similar types of locators. The fact that these tools use different syntaxes to encode test steps is immaterial where test breakages are concerned. Differences in the frequencies at which various locator strategies are employed, however, may indeed cause breakage data to differ. Nevertheless, we expect our taxonomy to generalize to other record/replay tools, possibly with different breakage frequencies.

C. Selection of Versions

To choose versions we relied on actual releases of our study objects. We chose releases as opposed to commits because releases more clearly represent checkpoints at which a given application is deemed ready for deployment and testing. Focusing on commits might threaten the validity of our study, by causing us to erroneously consider sets of changes that are not "complete" with respect to a developer's desired goal.

We began by considering all releases of each web application, but found that we did have to exclude some. These included early releases of applications that were unstable or no longer functioned on current platforms. We also excluded releases that contained serious faults. The numbers of releases listed in Table II are the numbers we ultimately retained.⁴

⁴For a list of the releases we considered, as well as pointers to the source code repositories for each object of study, see <https://sites.google.com/site/taxonomyobjects>.

³`cloc.sourceforge.net`

D. Test Creation and Execution Process

In searching for web applications, we discovered that few open-source applications are provided with capture-replay test suites; in fact, few are provided with any test suites at all. In the case of the applications we selected, Joomla had a suite of WebDriver tests, but only for its first version, and Dolibarr has a single WebDriver test. However, we have no knowledge as to whether the tests were created through a capture-replay approach, so using them was not a viable option. PHPAddressBook has unit tests that do not test the application's GUI. The other applications have no test suites. For this reason, we found it necessary to create new test suites.

To do this, we followed a systematic and iterative procedure for each web application. First, for each application the first author installed its initial version, V_0 , and familiarized herself with all of the functionalities provided by the application. Next, she constructed a test suite for V_0 . To minimize the subjectivity involved in this task, we adopted functional adequacy and partition/boundary value coverage criteria. To achieve functional adequacy, the first author created end-to-end use cases for the application, and continued to do so until all input fields and clickable items in each application had been exercised at least once. To achieve partition/boundary value coverage, she explicitly included in these use cases inputs exercising both nominal and exceptional behaviors, e.g., by choosing inputs, actions and boundary values that exercise non-mainstream application behavior, and by adding pauses to the input stream at appropriate moments to reflect user inactivity. Such use-case-based approaches to testing systems at the interface level are quite common in practice, and also well-suited to the use of record/replay tools.

Given a test suite T created for a version V (V_0 or a subsequent version), the first author executed that suite on the next version of the application, V' , and noted each case in which a test broke. She then manually repaired each of the tests that were repairable – an iterative process because repairing one test breakage might allow that test to proceed further and break again later. She also added new tests covering new functionality to the test suite. This resulted in a new test suite T' that now functioned on V' . The numbers of tests listed in Table II are the numbers we had available on the final version of each object of study.

We repeated the foregoing process for each version of the web application until each had been tested and the causes of all test breakages had been noted. (Again, this was applied in two phases, using the first five applications prior to taxonomy creation and the last three applications after it.) While following this process to repair and augment the tests for a given version V , we made no attempt to inspect the next version V' until after the repair and augmentation process was complete, reducing threats to validity that might result if tests explicitly “targeted” particular aspects of subsequent versions.

As one further note, all of the objects of study made use of databases, and in order to test them these databases needed to be populated with initial data; this was done by the first

author initially with the first version, and then, as application modifications necessitated, new versions of the databases were created appropriate to new application versions.

E. Taxonomization Process

After enumerating all causes of test breakages across the first five versions of our object web applications, we began the process of creating a taxonomy. To do this we adopted a systematic process. Essentially, this process involved clustering causes of test breakages in terms of similarity factors. For each test breakage that had been encountered, the first author studied it to determine its cause and wrote a description of it. Reviewing these descriptions allowed her to begin to identify candidate equivalence classes of test breakages, to which she assigned descriptive labels. These equivalence classes and labels were then reviewed with the second author and the two reached a consensus on them. The two authors then collaborated to organize the labels into hierarchies, again by using clustering based on the similarity of the factors responsible for the breakages.

For example, in version 7.00.06 of PHPFusion, a test uses a locator id “user_name” to locate an element (a text field to fill in). In the code of version 7.00.07, this id is changed to “AdminUsername”, rendering the test unable to locate the text field. The description of this breakage was: *The test broke when a locator attempted and failed to locate an element based on its attribute, “user_name”, because that attribute had been changed to “AdminUsername”*. The label for this was “Locator break due to failure to locate element attribute”. Later, review of these labels and consideration of other labels led us to rename this category of event “Element attribute not found”, to instantiate “Attribute based locators” as a parent category of this type of change, and to instantiate “Locators” as a parent of this.

Taxonomies such as ours are conceptual maps derived from empirical observations; as such they typically evolve as additional observations of the world are made. We expect the same to be true of our taxonomy, and thus, it is not necessarily the case that any attempt to apply the taxonomy to subsequent applications will allow every type of test breakage in those applications to be categorized. In such cases the taxonomy will require adjustments. That said, we believe that a useful taxonomy should handle the majority of new situations it encounters. This is why we conducted the second phase of data collection and taxonomy validation.

F. Threats to Validity

External validity threats concern the generalization of our findings. We considered only eight web applications and our results may not generalize to different ones. The selection criteria that we adopted, however, did ensure that these applications were of non-trivial size and had multiple versions, and had experienced many downloads and commits. Moreover, as we shall see, our validation phase provides at least initial evidence that the taxonomy is more widely applicable.

As a second threat, our results are gathered relative to tests created and executed using the Selenium IDE record/replay tool. As such, generalization to programmable (e.g., Selenium WebDriver) and visual (e.g., Sikuli) tools is not supported by our current results. Still, as noted earlier, many other record/replay tools do operate in manners similar to Selenium IDE, so we conjecture that the use of such tools might lead to similar results. Nevertheless, the only way to definitively address this threat is by further study.

As a third threat, it might also be argued that developers who do create tests will behave differently when modifying their applications, resulting in different frequencies of breakages in certain taxonomic categories. This too requires further study, but we do return to it in Section VI when discussing breakage avoidance.

Internal validity threats concern uncontrolled factors that may have affected the construction of the taxonomy. The first author created the tests. To control and reduce the subjectivity of this process, we defined precise adequacy criteria for test input creation and a precise procedure for test scenario construction; still, different tests might have revealed different breakages. The first author also classified the test breakages, participating with the second author. This task, however, requires reasoning that cannot be automated, so it is difficult to envision less threat-prone approaches. Moreover, to reduce the subjectivity involved in the task, she followed a systematic and structured procedure (write short breakage descriptions; assign labels to the descriptions; organize labels hierarchically), and interacted with the second author on the task.

Construct validity threats concern our metrics and measures. We have measured the numbers of test breakages observed in each taxonomic category, but this may not correlate with other important factors such as the difficulty of repairing or preventing (automatically or not) particular classes of breakages.

IV. TAXONOMY OF CAUSES OF TEST BREAKAGES

Before presenting our taxonomy, we comment on two issues: (a) the types of breakage causes being considered; and, (b) the granularity of breakages. Throughout this section, let V be a version of a web application, let t be a test created and executed on V , and let V' be a new version of V .

a) *Breakage causes*: Our taxonomy focuses on the *proximal* causes of test breakages; that is, the causes “most closely related to” each breakage that are found in the test code. Proximal causes are usually related to *distal* causes, such as the changes that were made to the web application code that resulted in the breakage. For example, a web application developer may modify the code of V by removing a particular element (e.g., a choice in a dropdown list) from that page. This modification may cause t to break in V' when it attempts to access the element. The fact that the locator corresponding to the dropdown list fails to point to any element is the proximal cause of the breakage; the fact that the developer modified the dropdown list is a distal cause.

Since we are interested in the evolution of test code, proximal causes are more relevant than distal causes. Thus,

our taxonomy classifies the causes of test breakage that most nearly impact the test code and does not attempt to categorize the types of changes associated with the evolution of the web application code. That said, when presenting examples of concrete test breakages corresponding to the categories in our taxonomy, we often find it convenient to mention distal causes as well.

b) *Granularity of breakages*: Tests can break multiple times, and we consider separately each distinguishable element of a test that acts as a proximal cause of a breakage. We have already noted in Section III that correcting one breakage and rerunning the test allows other breakages to appear later, and we count and categorize each such breakage. In addition, single statements or components of those statements can contain multiple causes of breakages. For example, suppose that V provides functionality allowing a user to change their password, via a text box having the id “password” in which the user enters their new password. Suppose that t contains the statement `type id=“password” mypassword`, which locates the “password” text box and types “mypassword” into it. Suppose that in V' , the id for the text box is capitalized (“Password”) and additional validation logic is added to the application code to verify whether the password is sufficiently complex (these modifications are distal causes). There are two separate breakages that may occur in this case: (1) t will break when it cannot locate the element attribute `id=“password”`, and (2) after that problem is corrected, t may break again if the password it enters is now invalid. Each of these breakages is a unique proximal cause that must be addressed separately, and we would count and categorize each of these individually.

Thus, to summarize, the data we present in this paper on test breakages pertain to proximal causes, and are collected with respect to all individual (“atomic” if you will) discrete proximal causes, whether these are located in a single Selenese statement or statement component, or not.

Fig. 3 presents a graphical view of our taxonomy of causes of test breakages. Nodes represent classes and subclasses of breakages, and edges represent complete, disjoint specialization relationships (i.e., given a node A that has edges to nodes B and C , A is a class of breakages that consists of two mutually exclusive subclasses, B and C). The numbers in non-root nodes uniquely identify each node while also identifying their chain of ancestors (if any). The numbers on edges are described in Section V.

At the highest level of our taxonomy we identified five distinct categories of causes of test breakages, as follows:

- 1) Causes related to locators used in tests.
- 2) Causes related to values and actions used in tests.
- 3) Causes related to page reloading.
- 4) Causes related to changes in user session times.
- 5) Causes related to popup boxes.

Each node within our taxonomy is related to one or more component within the three-tuple structure of a Selenese command `<command, target, value>` presented earlier in Section II.A. The first node of our taxonomy is related to locator

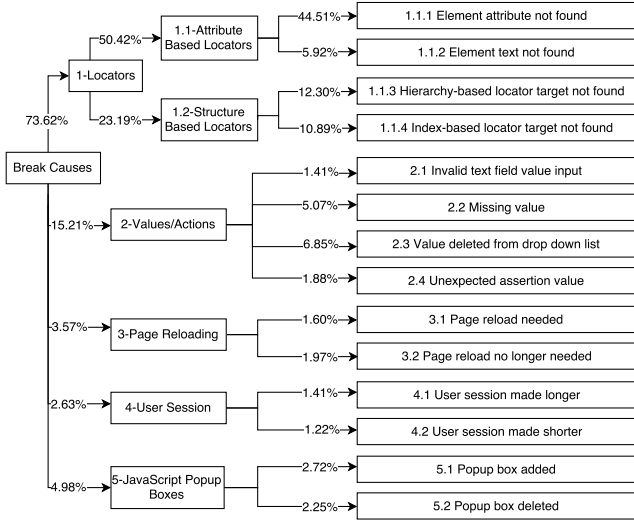


Fig. 3. Taxonomy

breakages. Locators are specified through the target component of a Selenese command. The second node of our taxonomy represents value and action breakages, which are also related to the value and action components of a Selenese command. The third node of our taxonomy accounts for page reloads. Page reloads are specified through the action component of a Selenese command (the action ClickAndWait is used in the case of a page reload, and the action Click is used in cases in which a page is not reloaded). The fourth node of our taxonomy is specific to user sessions and is related to two components within a Selenese command: action and value. Finally, the last node of our taxonomy is specific to popup boxes. The occurrence of popup boxes is represented through a Selenese statement specifying an action and a value. The action component represents the occurrence of the popup box (the AssertAlert action) and the value component accounts for the message displayed within the popup box.

We now discuss each of these categories in turn, in each case considering all subcategories beneath them.

1. Locators

The test breakage causes in this category are all proximally related to locators. Overall, this category has two subcategories, corresponding to the two major classes of locators described in Section II: attribute-based and structure-based.

1.1 Attribute-Based Locator Target not Found

As noted in Section II, elements of web pages have attributes and record/replay tests can make use of these to locate elements. They do this by identifying an element using its attribute's name/value pair or by looking for the element text, i.e., the HTML content that is visible in the user interface. We consider each of these access approaches separately.

1.1.1 Element Attribute not Found. This breakage is caused when t attempts to locate an element e via an attribute a that functioned (led to e) in V , but no longer functions in V' .

There are many potential distal causes of such a breakage. For example, a may have been modified. As a concrete instance, the test presented in Table I may break at Line 7 when attempting to locate the "Full Name" text field in the application if Line 21 of the HTML code shown in Fig. 2 is altered such that `name="FullName"` is now `name="Name"`. Deletions of attributes are also potential distal causes of such breakages. As an example involving status attributes, the test presented in Table I may break when attempting to locate the "checked" attribute in Line 2 of the test if the usage of that attribute is removed from Line 25 of the HTML code.

1.1.2 Element Text not Found. This breakage is caused when t attempts to locate an element e via its text, via a text string that functioned correctly (led to e) in V but no longer functions in V' . There are many potential distal causes of such a breakage. For example, the text in the element may have been modified. As a concrete instance, the test presented in Table I will break at Line 9 if the link text "Developed by M.Hammoudi" is changed.

1.2 Structure-Based Locator Target not Found

As noted in Section II, structure-based locators can be either hierarchy-based (utilizing the position of elements in the DOM tree), or index-based (utilizing indices), or both, and we distinguish between these two cases.

1.2.1 Hierarchy-Based Locator Target not Found. This breakage is caused when t attempts to locate an element e via a hierarchy-based locator l that functioned (led to e) in V , but no longer functions in V' . Distal causes of this breakage include the addition, deletion or modification of an ancestor of e in the DOM tree. For example, suppose that the "div" ancestor of the "Submit" button in the DOM tree for the application shown in Fig. 1 is deleted (by deletion of Lines 19 and 44 in the code). In this case the test presented in Table I will break at Lines 5 and (after partial repair) 8.

1.2.2 Index-Based Locator Target not Found. This breakage is caused when t attempts to locate an element e via an index-based locator l that functioned (led to e) in V , but no longer functions in V' . Distal causes of this breakage include the addition or deletion of elements to/from the web page that satisfy the same locator strategy of an element in V , but cause a change in the index of the element that t was intended to select. For example, suppose the text field for "Full Name" is removed from the application shown in Fig. 1. In this case the test presented in Table I will erroneously select the "Female" radio button at Line 2 and toggle that instead of the "Male" radio button, which is not the test's intended behavior. The test will then break upon reaching the `assertAlert` at Line 6, because the event that triggers that alert (absence of input to the "Full Name" text field) no longer occurs.

2. Values

All of the causes of breakages in this category are proximally related to values within tests.

2.1: Invalid Text Field Input. This breakage is caused when values used by t as inputs to V are no longer accepted in V' .

For instance, if t inputs a password in V , and in V' , restrictions on passwords are implemented that weren't present in V (e.g., required characters) then the password input by t may now be invalid on V' , in which case t will break.

2.2: Missing Value. This breakage is caused when values or actions not previously included in t as inputs to V are required in V' . For instance, a new text field or checkbox may be added to a form in V' , such that the form cannot be submitted unless an input is provided to the text field or an action is applied to the checkbox. As another example, this breakage can occur if a field or other page element for which inputs or actions are optional in V , and omitted from t , becomes required in V' .

2.3: Value Absent from Dropdown List.

This breakage is caused when a value that was previously selected by t from a dropdown list in V is no longer present in V' . For instance, a test of the web application presented in Fig. 1 may select the option "Fiat" from the "Car Brand" dropdown list; if that option is missing in a succeeding version of that application, the test will break.

2.4: Unexpected Assertion Value.

Assertions in record/replay tests can compare expected values to actual values, and signal cases in which these do not match. If the behavior of t with respect to an assertion differs when moving from V to V' , then t will break. This can occur, for example, if the text of an error message raised by the assertion changes in V' .

3. Page Reloading

The breakages within this category are all proximally related to page reloading activities. Page reloading may be triggered by user actions when, say, the current web page is updated in response to a user event. For example, in Fig. 1, if clicking on the "I live on campus" checkbox causes new fields such as "campus address" to be needed on the web page, a reload is needed. Like most record/replay tools, Selenese relies on a command (`clickAndWait`) to give the reload enough time to complete. There are two subcategories of these breakages.

3.1: Page Reload Needed. This breakage is caused if t , run on V , does not trigger a page reload, but does trigger a page reload when run on V' . In such cases, t may break due to the lack of a delay sufficient to allow its next action to succeed. Such a case may also arise if an action performed by t in V' causes the page to be reloaded in a longer time than was required in V , e.g., in order to present results of database queries that require longer wait times in the new version.

3.2: Page Reload no Longer Needed. This breakage is caused if t , run on V , does trigger a page reload, but does not trigger a page reload when run on V' . In such cases, t will attempt to wait for a page reload that will not occur, and will time out with an error message. Such a case could arise, for example, if Ajax facilities were introduced in V' to allow a partial page update in place of a full reload.

4. User Session Times

Many web applications utilize "timeouts" on user sessions; for example, a bank application may automatically log users out who are inactive for a certain amount of time. The breakages within this category are all proximally related to user session times. There are many ways in which timeouts may occur; for example, after m minutes of inactivity, a user may receive a popup message letting them know that they will be logged out in n minutes. User session limits may be encoded differently in tests depending on the manifestation of the session timeout. For instance, in the popup case, the test may contain statements that do the following: (1) wait for m minutes to simulate user inactivity, (2) issue an "assert Alert" announcing the impending logout. There are two subcategories of breakages related to user session times, as follows.

4.1: User Session Made Longer.

This breakage is caused if t , run on V , is meant to test (and wait for) a user session timeout of s_1 seconds, but when run on V' , it is required instead to test (and wait for) a user session timeout of s_2 seconds, where s_2 is sufficiently longer than s_1 . These breakages may be manifested in various ways; for instance, an alert JavaScript popup box containing the message "you will be logged out" may fail to appear in the time specified by the wait command, and the test will break.

4.2: User Session Made Shorter.

This breakage is caused if t , run on V , is meant to test (and waits for) a user session timeout of s_1 seconds, but when run on V' , it is required instead to test (and wait for) a user session timeout of s_2 seconds, where s_2 is shorter than s_1 . These breakages may be manifested in various ways; for instance, V' might automatically log the user out prior to the completion of the wait command, disabling any further actions by the test.

5. Popup Boxes

Popup boxes are often used in web applications to display error or other forms of messages. The breakages within this category are all proximally related to popups. Selenese relies on commands such as `assertAlert` to check whether popups function as expected. There are two subcategories of these breakages, as follows.

5.1: Popup Box Added.

This breakage is caused if t , run on V , did not expect a particular occurrence of some popup box B , but when run on V' it does encounter B . In this case, the test breaks due to the presence of the unexpected popup box. For example, in V' , the act of clicking on a submit button may trigger the appearance of a popup box containing the text "Thank you for your submission" that was not present in V .

5.2: Popup Box Deleted.

This breakage is caused if t , run on V , did expect a particular occurrence of some popup box B , but when run on V' , it does not encounter B . In this case, the test breaks due to the absence of the expected popup box. An example of this is the inverse of the example described for Category 5.1.

TABLE III
TEST BREAKAGE CAUSE COUNTS PER TAXONOMY CATEGORY AND WEB APPLICATION

	Training set					Test set			Total	%
	PHP-Fusion	Add-ress-Book	PHP-Agenda	My-Collaboration	Joomla	My-Movie-Library	Dolibarr	Your-Contacts		
1.1.1 Element attribute not found	45	63	31	53	138	38	52	54	474	44.51
1.1.2 Element text not found	10	7	3	8	15	4	9	7	63	5.92
1.2.1 Hierarchy-based locator target not found	22	16	14	16	11	12	14	15	131	12.30
1.2.2 Index-based locator target not found	39	6	12	11	11	13	7	17	116	10.89
2.1 Invalid text field value input	2	3	1	2	1	2	1	3	15	1.41
2.2 Missing value	4	5	6	3	18	5	4	9	54	5.07
2.3 Value deleted from drop down list	6	16	17	9	4	6	8	7	73	6.85
2.4 Unexpected assertion value	2	1	2	2	3	3	3	4	20	1.88
3.1 Page reload needed	2	1	1	1	2	2	4	4	17	1.60
3.2 Page reload no longer needed	2	2	1	1	6	2	2	5	21	1.97
4.1 User session made longer	3	1	3	1	1	1	3	2	15	1.41
4.2 User session made shorter	3	1	3	1	1	1	1	2	13	1.22
5.1 Popup box added	2	2	1	3	13	2	1	5	29	2.72
5.2 Popup box deleted	2	1	3	6	3	4	1	4	24	2.25
Total	155	125	98	117	227	95	110	138	1065	100.00

V. RESULTS

Table III lists the numbers of different causes of test breakages observed per web application and overall, for each of the leaf nodes in the taxonomy. We include data on all eight applications in the table, with the table header differentiating between the web applications utilized in the taxonomy construction phase (training set), and the applications utilized in the taxonomy validation phase (test set). The two rightmost columns of the table list the total numbers and percentages of causes of test breakages across all applications, per category, across all observed causes of breakages.

We have also annotated the taxonomy graph in Fig. 3, adding percentages to the edges. A percentage of $k\%$ on edge ($N1-N2$) indicates that $k\%$ of the total number (1065) of causes of test breakages are associated with that edge. These annotations show data pertaining to both leaf and non-leaf nodes; for example they show that, across all causes of test breakages, 73.62% involved locators, 15.21% involved values and actions, 3.57% involved page reloading, 2.63% involved user sessions, and 4.98% involved popup boxes.

The data on percentages of individual breakage categories show that a preponderance of breakages (44.51%) involved locator errors in which element attributes were not found, with the two types of structure-based locators being next (though far less) prevalent at 12.30% and 10.89%, respectively. Beyond locator problems, deletion of values from dropdown lists and missing values were the next most prevalent at 6.85% and 5.07%, respectively.

The data gathered on the applications in our test set largely mirror those gathered on applications in our training set, and the current taxonomic categories we created sufficed to categorize breakages in the test set. In particular, there was no single test breakage in the applications used for validation that could not be assigned to a category in the taxonomy, and no single category associated with breakages that did not occur in the additional web applications.

VI. DISCUSSION

Our taxonomy and associated data could be useful in several contexts related to test breakages.

Prioritization of breakages for test repair. Our data suggests which categories of test breakages merit the greatest attention. Locators caused over 73% of the test breakages we observed, and attribute-based locators caused the majority of these. Clearly, addressing just this class of errors by finding ways to repair them if they break would have the largest overall impact on the reusability of tests across releases. The data also suggest where subsequent priorities should be placed in terms of finding methods for repairing test breakages.

Prioritization of inspections. Knowledge about the causes of test breakages could help test developers manually repair tests in the absence of automated techniques. Given a broken test, they can prioritize the inspection of possible causes based on the relative importance of such causes in our taxonomy (e.g., by inspecting locators first, values next, etc.).

Breakage avoidance. One example of an avoidance-based approach to test breakages involves educating developers and maintainers of web applications as to the causes and probabilities of test breakages. Such education could be supported by information present in our taxonomy and the data that underlie it, which could serve as a source of guidelines on best practices to follow while changing web application code. For example, we observed that many of the breakages involving attribute-based locators resulted from simple name changes, i.e., changing an attribute from `name="submitButton"` to `name="SubmitButton"`. While such name changes may be helpful for code readability, their cost in terms of impact on tests can be quite high.⁵

⁵In our discussion of threats to validity (Section III.F) we noted that developers who use capture-replay tests may be well aware that they should avoid changing attribute locators since they are likely to know that this can break tests. We did find, however, that on Joomla, where WebDriver tests were provided, there was still a large number of test breakages related to changes in attribute locators, as shown in Table III. Further, in organizations in which testing is performed by persons other than developers, the incentive to avoid changes may not be strong.

Breakage prevention. Knowledge about causes of test breakages could also help web application and test developers prevent them. For instance, they may introduce the practice of adding meaningful and stable `id`'s to the core elements of the web interface, or they may take advantage of tools like ATA [45] or ROBULA [21] to create robust locators automatically. Another way to produce change-resilient tests is by adopting design patterns such as the *Page Object*⁶ pattern. A Page Object acts as a mediator between a test and the web page whose elements it references. Such a mediator hides the internal, implementation details of the web page DOM from the test code by exposing only abstract interaction functions. When a breakage occurs, only the affected Page Object needs to be fixed; all tests using it will be repaired at the same time. Approaches such as this, however, are not yet widespread. Our data could help motivate their adoption.

Bad smells in web tests. Many techniques exist for statically analyzing code to locate “bad smells” [3] – stylistic problems or malfeasances that developers may wish to guard against. Guided by our taxonomy, analogous tools could analyze web applications and their tests for potential problems. For instance, analyses could detect the absence of stable anchors for DOM elements that are expected to be important during testing (e.g., a `div` tag, showing the results produced after a form submission that are potentially useful to create test assertions). Where test breakages are concerned, however, such analyses might more appropriately be directed at pairs of versions of web applications, because it is the changes made to applications that are the distal causes of breakages and need to be considered. For instance, a DOM element that caused repeated test breakages in the past might deserve a dedicated localization mechanism (e.g., a unique `id` or name).

IDE enhancements. An “after-the-changes-are-made” approach to analysis may be valuable, but it seems likely that an analysis approach that operates concurrently with program maintenance might be even more effective, and such an analysis could also be guided by our taxonomy. Modern IDEs for code development typically employ such approaches: as programmers edit, they point out problems or provide useful information based on the programmers’ actions. Web application development IDEs employ such approaches also, but to our knowledge, no such IDEs also build in assistance related to testing efforts. Such IDEs could aid in test breakage avoidance by alerting programmers to possible effects and letting them choose whether or not to act on them. They could aid in prevention by prohibiting certain actions or by recommending the creation of stable anchors for locators whenever they recognize that locators used in tests are likely to be fragile. They could maintain information on changes made to a web application, that testers could later use when reviewing tests in preparation for applying them to a new version (a notion also suggested by Daniel et al. [10]). Such information could also be utilized by automated test repair techniques themselves. In that context, instead of postponing

test repair efforts until later, such IDEs might even be capable of programmatically correcting tests in response to particular edits by refactoring test code related to locators, values, and so forth as the code is edited.

Root cause analysis. A final issue pertains to identifying test breakages and locating (or helping engineers locate) the proximal causes of those breakages. As we classified the causes of test breakages based on actual data, it became quite clear that the time at which the proximal cause of a breakage occurs and the time at which a breakage is detected can differ widely. *Direct breakages* manifest themselves precisely when the breakage cause is encountered, as for example when a popup box is added or deleted (taxonomy categories 5.1 and 5.2). *Propagated breakages* do not manifest themselves immediately, but do manifest themselves later on subsequent test actions. For example, a specific attribute value a may be used in two different elements e_1 and e_2 of web application V , and test t may use a as a locator. In a version V' of the web application, if the use of a in e_1 has been deleted, t will refer to the use of a in e_2 instead. Depending on the result of this action, t may continue for some time before encountering a situation in which its next action cannot be completed. *Silent breakages* never manifest themselves explicitly. The previous example functions here as well, if t remains able to continue through until completion. In this case, t can still be utilized as a test, but from a test design standpoint this is quite problematic because t is not now testing what it was designed to test. These examples suggest that in addition to preventing, avoiding, or repairing test breakages, we need better methods for detecting them and for tracing them back to the problematic test code that serves as their proximal cause. Methodologies that monitor test execution for problems, perhaps similar to those created to monitor web macro execution for problems [19] might be effective here.

VII. RELATED WORK

There has been some research on automated techniques for repairing *programs* (e.g., [8], [14], [26], [31], [42]–[44]). Our work focuses on tests.

There have been numerous papers on test repair. Several papers have addressed the problem of repairing unit tests such as JUnit tests or tests written in similar frameworks (e.g., [10], [11], [30]). Such approaches, however, are not applicable to tests produced by means of record/replay tools.

Many papers have addressed the problem of repairing GUI tests. GUI tests often track sequences of user actions applied to an interface, and some of these approaches may be adaptable to web applications and record/replay tests. Memon and Soffa [28], Memon [27] and Datchayani et al. [13] use event flow graphs and transformation techniques to repair broken GUI tests. Huang et al. [17] use a genetic algorithm to repair GUI tests. Grechanik et al. [15] present an approach that analyzes an initial and modified GUI for differences and generates a report for engineers documenting ways in which associated test scripts may be broken by changes. Daniel et al. [12] use GUI refactorings to keep track of changes

⁶<https://code.google.com/p/selenium/wiki/PageObjects>

engineers make to a GUI, suggesting that this information could later be used to repair tests.

Zhang et al. [47] address the problem of repairing broken *workflows* in GUI applications, where a workflow is a sequence of activities to perform a given task. This approach, however, does not target tests.

Alshawan and Harman [1] present an approach for repairing user session data used to regression test web applications. User session data, however, differ from the data captured by record/replay tools; it records just the requests received by a server from a web application.

Four recent papers consider problems related to record/replay tests of web applications. Stocco et al. [40] investigate the automated generation of page objects that confine causes of test breakages to a single class, a form of breakage prevention. Yandrapally et al. [45] address the problem of test script fragility in relation to locators, proposing approaches for identifying UI elements in more robust ways using contextual clues, also a form of prevention. Another approach for producing robust locators has been implemented in the tool ROBULA [21]. The multi-locator extension of ROBULA [22] supports automated repair of broken locators. Choudhary et al. [9] focus, as do we, on test scripts such as those created by Selenium, and present a technique that based on differential testing that compares the behavior of a broken test to its behavior on a prior version of the web application. Based on the differences, the technique suggests potential test repairs.

Researchers have presented various fault taxonomies. Bruning et al. [4] present a fault taxonomy for service-oriented architectures. Hayes [16] provides a fault taxonomy for NASA software requirements. Mariani [25] presents a taxonomy for component based software systems. Chan et al. [7] provide a taxonomy for web service compositions. Hummer et al. [18] present a taxonomy for event-based systems. None of these taxonomies consider web applications.

A few papers present fault taxonomies for web applications. Ocariza et al. [32], [33] characterize the classes of error messages output by JavaScript in web applications and the classes of faults found in JavaScript. Ricca and Tonella [35] present a preliminary fault taxonomy for web applications, consisting of a single level of general fault categories. Marchetto et al. [24] present a taxonomy of web application faults. None of these papers consider test breakages.

Leotta et al. [20] conducted an empirical study in which they measured the effort associated with the evolution of two equivalent Selenium IDE and Selenium WebDriver test suites, with the aim of comparing the maintainability of record/replay vs. programmable test suites. Changes made to the tests were classified as logical or structural. However, no further refinement of this classification was carried out.

To our knowledge, the only paper that provides a more detailed classification of the ways in which tests of web applications may break is the test repair paper by Choudhary et al. [9], described earlier. The authors identify three types of changes related to broken test scripts. (1) Structural changes are changes in the DOM tree and may cause locator problems

that can be characterized as “non-selection” or “mis-selection” problems. One type of structural change involves additions, deletions or modifications of form elements and these comprise the “form data problem”. (2) Content changes involve modifications of the text or HTML contained in a DOM node; these can affect assertions that check node contents, and lead to “obsolete content” problems. (3) Blind changes involve changes in server-side code. This classification focuses first on changes and then on associated problems that may result in test breakages, whereas we focus on breakages. Further, while based on the authors’ experience, the classification is not drawn from any formal empirical study of web applications, whereas ours is. Finally, our taxonomy recognizes several types of breakages not noted by Choudhary et al., handling blind changes by focusing on proximal causes, which involve events or actions occurring on the client side.

VIII. CONCLUSIONS AND FUTURE WORK

Following a rigorous process based on empirical observation of hundreds of versions of non-trivial, popular web applications, we have constructed a taxonomy of the causes of breakages of tests created by record/replay approaches for web applications, and the frequency at which the identified causes contribute to test breakages. We believe that our taxonomy can help practitioners create less brittle tests, and help researchers find better ways to repair tests, prevent and avoid breakages, and create IDEs and record/replay tools that are more robust relative to test breakages.

Our taxonomy presents a quantitative and a qualitative assessment of the causes behind test breakages using Selenium IDE, which is a second generation testing tool. Second generation tools identify web elements to be selected within the user interface according to their position within the DOM tree and/or attributes. There are, however, two other generations of tools that have been used to test web applications. First generation tools rely on screen coordinates in order to locate and manipulate web elements within the web application under test, and have largely been deprecated due to the flakiness of coordinates. Third generation tools test web applications based on image recognition of web elements within the user interface of the web application under test. As future work, we plan to investigate test breakage taxonomies for third generation tools and other second generation tools (e.g., Selenium WebDriver).

As noted earlier, taxonomies do typically evolve as additional observations of the world are made. While our initial attempts to validate our taxonomy revealed no shortcomings, it is likely that continued studies will reveal the need for adjustments. Finally, as noted, there are several ways in which our results can be used to direct research on new ways to improve record/replay testing and reduce test breakages.

ACKNOWLEDGEMENTS

This work has been partially supported by the National Science Foundation through award IIS-1314365. We thank Wayne Motycka for his assistance with study infrastructure and data analysis.

REFERENCES

- [1] N. Alshawan and M. Harman. Automated session data repair for web application regression testing. In *International Conference on Software Testing, Verification, and Validation*, pages 298–307, 2008.
- [2] S. Andrica and G. Candea. WaRR: A tool for high-fidelity web application record and replay. In *DSN*, pages 403–410, June 2011.
- [3] G. Bavota, A. Qusef, R. Oliveto, A. De Lucia, and D. Binkley. Are test smells really harmful? An empirical study. *Empirical Software Engineering Journal*, 20:1052–1094, 2015.
- [4] S. Bruning, S. Weissleder, and M. Malek. A fault taxonomy for service-oriented architecture. In *IEEE High Assurance Systems Engineering Symposium*, pages 367–368, 2007.
- [5] B. Burg, R. Bailey, A. J. Ko, and M. D. Ernst. Interactive record/replay for web application debugging. In *ACM User Interface Software and Technology Symposium*, pages 473–484, 2013.
- [6] CasperJS. casperjs.org.
- [7] K. S. M. Chan, J. Bishop, J. Steyn, L. Baresi, and S. Guinea. A fault taxonomy for web service composition. In Elisabetta Di Nitto and Matei Ripeanu, editors, *Service-Oriented Computing - ICSOC 2007 Workshops*, volume 4907 of *Lecture Notes in Computer Science*, pages 363–375. Springer, 2009.
- [8] S. Chandra, E. Torlak, S. Barman, and R. Bodik. Angelic debugging. In *International Conference on Software Engineering*, pages 121–130, 2011.
- [9] S. R. Choudhary, D. Zhao, H. Versee, and A. Orso. WATER: Web Application TEst Repair. In *First International Workshop on End-to-End Test Script Engineering*, pages 24–29, 2011.
- [10] B. Daniel, T. Gvero, and D. Marinov. On test repair using symbolic execution. In *International Symposium on Software Testing and Analysis*, pages 207–218, 2010.
- [11] B. Daniel, V. Jagannath, D. Dig, and D. Marinov. ReAssert: Suggesting repairs for broken unit tests. In *International Conference on Automated Software Engineering*, pages 433–444, 2009.
- [12] B. Daniel, Q. Luo, M. Mirzaaghaei, D. Dig, D. Marinov, and M. Pezze. Automated GUI refactoring and test script repair. In *First International Workshop on End-to-End Test Script Engineering*, pages 38–41, 2011.
- [13] M. Dhatchayani, X. A. R. Arockia, P. Yogesh, and B. Zacharias. Test case generation and reusing test cases for GUI designed with HTML. *Journal of Software*, 7(10):2269–2277, 2012.
- [14] D. Gopinath, M. Z. Malik, and S. Khurshid. Specification-based program repair using SAT. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 173–188, 2011.
- [15] M. Grechanik, Q. Xie, and C. Fu. Maintaining and evolving GUI-directed test scripts. In *International Conference on Software Engineering*, pages 408–418, 2009.
- [16] J. H. Hayes. Building a requirement fault taxonomy: Experiences from a NASA verification and validation research project. In *International Symposium on Software Reliability Engineering*, pages 49–60, 2003.
- [17] S. Huang, M. B. Cohen, and A. M. Memon. Repairing GUI test suites using a genetic algorithm. In *International Conference on Software Testing*, pages 245–254, 2010.
- [18] W. Hummer, C. Inzinger, P. Leitner, B. Satzger, and S. Dustdar. Deriving a unified fault taxonomy for event-based systems. In *ACM International Conference on Distributed Event-Based Systems*, pages 167–178, July 2012.
- [19] A. Koesnandar, S. Elbaum, G. Rothermel, L. Hochstein, K. Thomasset, and C. Scaffidi. Using assertions to help end-user programmers create dependable web macros. In *ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 124–134, 2008.
- [20] M. Leotta, D. Clerissi, F. Ricca, and P. Tonella. Capture-replay vs. programmable web testing: An empirical assessment during test case evolution. In *Working Conference on Reverse Engineering*, pages 272–281, 2013.
- [21] M. Leotta, A. Stocco, F. Ricca, and P. Tonella. Reducing web test cases aging by means of robust xpath locators. In *IEEE International Symposium on Software Reliability Engineering Workshops*, pages 449–454, 2014.
- [22] M. Leotta, A. Stocco, F. Ricca, and P. Tonella. Using multi-locators to increase the robustness of web test cases. In *IEEE International Conference on Software Testing, Verification and Validation*, pages 1–10, 2015.
- [23] G. Leshed, E. M. Haber, T. Matthews, and T. Lau. Coscripter: Automating & sharing how-to knowledge in the enterprise. In *ACM Conference on Human Factors in Computing Systems*, pages 1719–1728, 2008.
- [24] A. Marchetto, F. Ricca, and P. Tonella. Empirical validation of a web fault taxonomy and its usage for fault seeding. In *International Workshop on Web Site Evolution*, pages 31–38, 2007.
- [25] L. Mariani. A fault taxonomy for component-based software. In *International Workshop on Test and Analysis of Component-Based Systems*, pages 55–65, 2003.
- [26] S. Mechtaev, J. Yi, and A. Roychoudhury. DirectFix: Looking for simple program repairs. In *International Conference on Software Engineering*, pages 448–458, 2015.
- [27] A. M. Memon. Automatically repairing event sequence-based GUI test suites for regression testing. *ACM Transactions on Software Engineering and Methodology*, 18(2):4:1–4:36, 2008.
- [28] A. M. Memon and M. L. Soffa. Regression testing of GUIs. In *International Symposium on Foundations of Software Engineering*, 2003.
- [29] J. Mickens, J. Elson, and J. Howell. Mugshot: Deterministic capture and replay for JavaScript applications. In *USENIX Conference on Networked Systems Design and Implementation*, 2010.
- [30] M. Mirzaaghaei, F. Pastore, and M. Pezze. Supporting test suite evolution through test case adaptation. In *International Conference on Software Testing, Verification and Validation*, pages 231–240, 2012.
- [31] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra. Semfix: Program repair via semantic analysis. In *International Conference on Software Engineering*, pages 772–781, 2013.
- [32] F. Ocariza, K. Bajaj, K. Pattabiraman, and A. Mesbah. An empirical study of client-side JavaScript bugs. In *International Symposium Empirical Software Engineering and Measurement*, pages 55–64, 2013.
- [33] F. Ocariza, K. Pattabiraman, and B. Zorn. JavaScript errors in the wild: An empirical study. In *International Symposium on Software Reliability Engineering*, pages 100–109, 2011.
- [34] TestMaker Object Designer. www.pushotest.com/designer.
- [35] F. Ricca and P. Tonella. Web testing: A roadmap for empirical research. In *International Symposium on Web Site Evolution*, 2007.
- [36] Sahi. sahipro.com.
- [37] The Selenium Project. http://seleniumhq.org/docs/03_webdriver.html/.
- [38] Watir WebDriver. <http://watirwebdriver.com>.
- [39] K. Sen, S. Kalasapur, T. Brutch, and S. Gibbs. Jalangi: A selective record-replay and dynamic analysis framework for JavaScript. In *International Symposium on Foundations of Software Engineering*, pages 488–498, 2013.
- [40] A. Stocco, M. Leotta, F. Ricca, and P. Tonella. Why creating web page objects manually if it can be done automatically? In *IEEE/ACM International Workshop on Automation of Software Test*, pages 70–74, 2015.
- [41] Unified Functional Testing (UFT). <http://www8.hp.com/us/en/software-solutions/unified-functional-automated-testing>.
- [42] Y. Wei, Y. Pei, C. A. Furia, L. S. Silva, S. Buckholz, B. Meyer, and A. Zeller. Automated fixing of programs with contracts. In *International Symposium on Software Testing*, pages 61–72, 2010.
- [43] W. Weimer, S. Forrest, C. Le Goues, and T. Nguyen. Automatic program repair with evolutionary computation. *Communications of the ACM*, 53(5):109–116, 2010.
- [44] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest. Automatically finding patches using genetic programming. In *International Conference on Software Engineering*, pages 364–374, 2009.
- [45] R. Yandrapally, S. Thummalapenta, S. Sinha, and S. Chandra. Robust test automation using contextual clues. In *International Symposium on Software Testing and Analysis*, pages 304–314, 2014.
- [46] T. Yeh, T.-H. Chang, and R. C. Miller. Sikuli: Using GUI screenshots for search and automation. In *ACM User Interface Software and Technology Symposium*, pages 183–192, 2009.
- [47] S. Zhang, H. Lü, and M. D. Ernst. Automatically repairing broken workflows for evolving GUI applications. In *International Symposium on Software Testing and Analysis*, pages 45–55, 2013.